

HR Employee Attrition Project Code Explanation

This document explains the files currently present in the project folder. It is based on the actual code in `streamlit_app.py` and `hr_attrition_dashboard.html`.

Important accuracy note: The Streamlit file trains real scikit-learn models. The HTML file does not train a machine-learning model; it uses a browser-side risk scoring formula based on dataset rates and selected input conditions.

1. Project Files

File	Purpose
<code>Balanced_HR_Employee_Attrition.csv</code>	Main HR dataset. The target column is <code>Attrition</code> , with values <code>Yes</code> and <code>No</code> .
<code>streamlit_app.py</code>	Python Streamlit application. It loads the CSV, displays EDA charts, trains a selected model, shows metrics, and predicts attrition for one employee profile.
<code>hr_attrition_dashboard.html</code>	One-page HTML, CSS, and JavaScript dashboard. It loads the CSV in the browser, shows charts and filters, and calculates a live risk score.
<code>PROJECT_DOCUMENTATION.md</code>	Short project documentation with run steps and feature summary.

2. Dataset Target and Dropped Columns

Both applications use `Attrition` as the target concept. In the Streamlit ML pipeline, the target is converted as follows:

```
y = df[TARGET].map({"No": 0, "Yes": 1})
```

The Streamlit app also defines these columns as dropped feature columns:

```
DROP_COLUMNS = ["EmployeeNumber", "EmployeeCount", "StandardHours", "Over18"]
```

These columns are removed from the model feature matrix because they are identifiers or constant/non-informative fields in this project code.

3. Streamlit App Explanation

3.1 Imports

The Streamlit app imports:

- `pathlib` for locating the CSV beside the Python file.
- `numpy` and `pandas` for data handling.
- `plotly.express` and `plotly.figure_factory` for charts.
- `streamlit` for the web app UI.
- scikit-learn classes for preprocessing, model training, train/test split, and metrics.

3.2 Page Setup and Styling

`st.set_page_config()` sets the page title, icon text, wide layout, and expanded sidebar. The app also injects custom CSS through `st.markdown(..., unsafe_allow_html=True)` to style the hero section, metrics, and dataframe borders.

3.3 `load_data()`

This function is decorated with `@st.cache_data(show_spinner=False)`. It reads `Balanced_HR_Employee_Attrition.csv` using `pd.read_csv(DATA_PATH)`. The cache prevents repeated file reads during normal Streamlit reruns.

3.4 `split_columns(df)`

This function removes the target and dropped columns, then separates remaining columns into numeric and categorical lists:

- Numeric columns are detected with `select_dtypes(include=np.number)`.
- Categorical columns are detected with `select_dtypes(exclude=np.number)`.

3.5 `build_model(model_name, numeric_cols, categorical_cols)`

This function builds a scikit-learn `Pipeline`. It uses a `ColumnTransformer` with:

- `StandardScaler()` for numeric features.
- `OneHotEncoder(handle_unknown="ignore", sparse_output=False)` for categorical features.

The estimator depends on the selected model name:

Model option	Estimator used in code
Random Forest	<code>RandomForestClassifier(n_estimators=250, max_depth=10, min_samples_leaf=2, random_state=42, n_jobs=-1)</code>

Gradient Boosting	<code>GradientBoostingClassifier(random_state=42)</code>
Logistic Regression	<code>LogisticRegression(max_iter=2000, class_weight="balanced", random_state=42)</code>

3.6 `train_model(model_name, test_size)`

This cached function performs the ML workflow:

1. Loads the dataset.
2. Gets numeric and categorical columns.
3. Creates `X` by dropping the target and the dropped columns.
4. Creates `y` by mapping `No` to `0` and `Yes` to `1`.
5. Splits data using `train_test_split(..., random_state=42, stratify=y)`.
6. Builds and fits the selected pipeline.
7. Predicts test labels and probabilities.
8. Returns the fitted pipeline, test data, column lists, metrics, and classification report.

The metrics calculated in the code are `Accuracy`, `Precision`, `Recall`, and `ROC AUC`.

3.7 `attrition_rate(data, group_col)`

This helper creates an `AttritionFlag` column where `Yes` becomes `1`. It groups by the requested column and calculates:

- `AttritionRate`: mean of `AttritionFlag`, converted to percentage.
- `Employees`: row count for each group.

3.8 `plot_eda(df)`

This function displays the Streamlit exploratory data analysis section. It includes six main charts:

1. Overall attrition balance pie chart.
2. Attrition rate by department bar chart.
3. Age distribution by attrition histogram.
4. Monthly income spread by attrition box plot.
5. Overtime versus attrition rate bar chart.
6. Correlation heatmap of selected numeric columns.

It also includes an optional deep-dive selector for `JobRole`, `EducationField`, `BusinessTravel`, `MaritalStatus`, and `Gender`.

3.9 `feature_importance(model_result)`

This function returns feature importance only when the trained estimator has a `feature_importances_` attribute. This applies to Random Forest and Gradient Boosting, but not Logistic Regression in the current code. It uses

`preprocessor.get_feature_names_out()` to align transformed feature names with importance values.

3.10 `plot_model_results(model_result)`

This section shows four metric cards, a confusion matrix, and either feature importance or the classification report. If feature importance is unavailable, it displays the report as a dataframe.

3.11 `prediction_form(df, model_result)`

This function builds the employee prediction form. It collects user values for profile, work, and satisfaction inputs. For any model feature not directly shown in the form, it fills a default value:

- Numeric missing inputs use median values.
- Categorical missing inputs use the mode value.

It then creates a one-row dataframe with the same columns as the model feature matrix and calls:

```
model_result["pipeline"].predict_proba(input_df)[0, 1]
```

The resulting probability is displayed as the attrition risk.

3.12 `main()`

The `main()` function connects everything together. It loads data, builds sidebar controls, shows summary metrics, displays EDA, trains the selected model, shows model results, and renders the prediction form.

4. HTML Dashboard Explanation

4.1 File Structure

`hr_attrition_dashboard.html` is a single-page file containing HTML, CSS, and JavaScript. It does not depend on an external JavaScript chart library. Charts are drawn manually on `canvas` elements.

4.2 Main Page Sections

Section	What it contains
Hero section	Title, short explanation, navigation buttons, and four metric cards.
<code>eda</code> section	Filters and six canvas charts.
<code>predict</code> section	Prediction form and live risk result card.

Filtered employee rows	Table preview of rows matching selected dashboard filters.
------------------------	--

4.3 Dashboard Filters

The filter controls are:

- `filterDepartment`
- `filterJobRole`
- `filterOverTime`
- `filterAttrition`

The `applyFilters()` function filters `allRows` into `rows`, then refreshes statistics, metrics, charts, and the table.

4.4 CSV Loading

The JavaScript defines:

```
const csvPath = "Balanced_HR_Employee_Attrition.csv";
```

The `init()` function uses `fetch(csvPath)`, parses the CSV text with `parseCSV()`, stores the complete dataset in `allRows`, and initializes dashboard state.

4.5 CSV Parser

`parseCSV(text)` is a custom parser. It handles quoted values and escaped quotes. It converts numeric-looking cell values into JavaScript numbers and leaves other values as strings.

4.6 Statistics Helpers

Function	What it does
<code>groupBy(data, column)</code>	Groups row objects by a selected column.
<code>riskRate(data)</code>	Returns the share of rows where <code>Attrition === "Yes"</code> .
<code>rateBy(column)</code>	Calculates attrition rate and row count for each value in a column.
<code>average(column)</code>	Calculates the mean of a numeric column for the current <code>rows</code> .
<code>median(column)</code>	Calculates the median of a numeric column for the current <code>rows</code> .
<code>buildStats(data)</code>	Builds baseline rate, averages, median income, and categorical rate tables.

4.7 Metric Cards

`setupMetrics()` updates the four top cards:

- Employees shown.
- Leave rate.
- Average monthly income.
- Overtime risk ratio, calculated when both overtime groups exist in the filtered data.

4.8 Chart Drawing

The HTML dashboard draws charts with canvas functions:

Function	Purpose
<code>clearCanvas(canvas)</code>	Clears and scales the canvas for the current device pixel ratio.
<code>drawNoData(canvasId)</code>	Shows a no-data message when filters remove all matching rows.
<code>drawAxes(ctx, w, h, pad)</code>	Draws chart axes.
<code>drawBars(...)</code>	Draws vertical or horizontal bar charts.
<code>drawDonut(...)</code>	Draws the stayed-versus-left donut chart.
<code>histogram(column, bins)</code>	Creates binned counts for numeric columns.
<code>drawLine(...)</code>	Draws age distribution bars and an attrition-rate line.
<code>drawCharts()</code>	Calls all chart drawing functions for the six chart panels.

4.9 HTML Prediction Form

The HTML prediction form contains inputs for age, department, job role, monthly income, overtime, business travel, distance from home, working years, years at company, job satisfaction, work-life balance, marital status, education field, stock option level, and salary hike percentage.

`setupForm()` fills select boxes from dataset values where applicable. It also attaches `input`, `change`, and `keyup` listeners to every form field, so `updatePrediction()` runs whenever a value changes.

4.10 HTML Prediction Logic

The HTML dashboard prediction is a risk scoring method, not a trained model. The calculation starts from `modelStats.baseRate`, the overall attrition rate from the full dataset. Then the code adjusts the score using:

- `categoryAdjustment(...)`, which compares the selected category's attrition rate with the baseline rate.
- `numericAdjustment(...)`, which adds or subtracts fixed risk amounts when conditions are true.

Examples of numeric or condition-based adjustments in the current code include overtime, frequent travel, low or high satisfaction, low or high work-life balance, age below 30, age above 48, lower or higher monthly income, longer commute, short or long tenure, stock option level, and salary hike percentage.

The final score is bounded between `0.03` and `0.96` :

```
const bounded = Math.max(0.03, Math.min(0.96, score));
```

The page then updates `riskNumber`, `riskFill`, `riskLabel`, `riskUpdated`, and `riskDrivers`.

4.11 Dataset Preview Table

`renderTable()` displays selected columns from the current filtered `rows`. If no rows match selected filters, it shows a no-row message.

5. How to Run

5.1 Streamlit App

```
streamlit run streamlit_app.py
```

Then open the local Streamlit URL shown by Streamlit.

5.2 HTML Dashboard

```
python -m http.server 8010 --bind 127.0.0.1
```

Then open:

```
http://127.0.0.1:8010/hr_attrition_dashboard.html
```

The HTML page should be opened through a local server because it fetches the CSV file. Opening the file by double-clicking may block the CSV fetch in some browsers.

6. Key Limitations Stated Clearly

- The Streamlit app uses real ML training through scikit-learn.
- The HTML dashboard uses a heuristic risk score and does not train or load a scikit-learn model.
- The HTML charts are custom canvas drawings, not Plotly or Chart.js charts.
- The HTML dashboard reads the CSV at runtime using `fetch()`.

- The Streamlit app calculates metrics on a train/test split selected from the sidebar.

7. Function Index

7.1 Streamlit Functions

`load_data`, `split_columns`, `build_model`, `train_model`, `attrition_rate`, `plot_eda`, `feature_importance`, `plot_model_results`, `prediction_form`, and `main`.

7.2 HTML JavaScript Functions

`parseCSV`, `pct`, `money`, `groupBy`, `riskRate`, `rateBy`, `average`, `median`, `buildStats`, `computeStats`, `setupMetrics`, `fillSelect`, `unique`, `fillFilter`, `setupFilters`, `applyFilters`, `setupForm`, `categoryAdjustment`, `numericAdjustment`, `updatePrediction`, `clearCanvas`, `drawNoData`, `drawAxes`, `drawBars`, `drawDonut`, `histogram`, `drawLine`, `drawCharts`, `renderTable`, and `init`.